

Torra & Terra

Do documento ao deploy

O mesmo e-commerce do projeto 1, agora num banco de documentos — e o que isso muda.

Prof. Nivaldo T. Marcusso · [PREENCHER: nomes do grupo]

Como trabalhamos

Um plano escrito antes. Depois, uma etapa por vez — cada uma com commit e testes.

0

Requisitos e consultas

o que a loja pergunta ao banco

1

Modelo de fichas

embutir, referenciar, fotografar

2

CouchDB e índices

regras no banco e 4 índices

3

Aplicação

o Flask falando HTTP com o banco

4

Checkout

a saga, com estorno

5

Testes e conflito

o 409 forçado de propósito

6

Deploy e documentação

Vercel, Cloudant e os documentos

O plano foi escrito em 9 de setembro, antes de qualquer código. A construção teve o Claude Code como parceiro de bancada, e cada etapa entrou no Git com os testes rodando.

Por que NoSQL aqui

Escolhemos pelo jeito que a loja usa os dados — o que o site mais pergunta ao banco.

O catálogo é lido o tempo todo

Toda visita pede a lista de cafés. Com um índice pronto para essa pergunta, a resposta sai sem juntar tabelas.

O pedido é lido inteiro, sempre

Pedido e itens nunca aparecem separados — então cabem numa ficha só, lida de uma vez.

O café tem atributos que mudam

Nota sensorial, pontuação, moagem: uma ficha aceita um campo novo sem reformar o arquivo inteiro.

AS PERGUNTAS, LISTADAS ANTES DE DESENHAR AS FICHAS

Quais cafés estão à venda?	<code>idx_produtos_catalogo</code>
----------------------------	------------------------------------

Quais cafés desta região?	<code>idx_produtos_categoria</code>
---------------------------	-------------------------------------

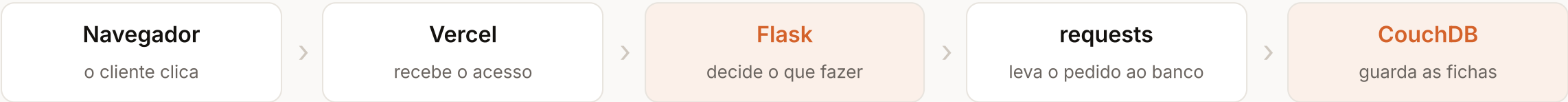
Quem é o cliente deste e-mail?	<code>idx_clientes_email</code>
--------------------------------	---------------------------------

Quais são os pedidos deste cliente?	<code>idx_pedidos_cliente</code>
-------------------------------------	----------------------------------

O que se perde: o “tudo ou nada” entre fichas diferentes. O slide 8 mostra como resolvemos.

Como o site funciona por dentro

O caminho que um clique percorre — do navegador até a ficha guardada no banco.



E na volta, o Jinja encaixa a resposta na página que o cliente vê.

O QUE É CADA PEÇA, EM UMA FRASE

Vercel	nuvem	É o PORTEIRO . Recebe cada acesso da internet e acorda o Python para atender.
Flask	Python	É o CÉREBRO . Decide o que responder, e é onde moram as regras.
Jinja	HTML	É o MOLDE da página. Encaixa o dado no HTML que o cliente vê.
requests	Python	É o CARTEIRO . Leva cada pedido ao CouchDB por HTTP, como se o banco fosse um site.
CouchDB	JSON	É o ARQUIVO DE FICHAS . Cada café, cliente e pedido é uma ficha completa, guardada inteira.
Cloudant	JSON	É o mesmo arquivo, guardado num prédio da IBM: o CouchDB rodando na nuvem.

Não há tradutor no meio: o CouchDB fala HTTP, e cada leitura no banco é como abrir uma página.

Ferramentas

O que usamos em cada etapa, e para quê.

MODELAGEM E BANCO

Apache CouchDB 3.5

o arquivo de fichas

IBM Cloudant

o mesmo arquivo na nuvem, plano gratuito

Mango e _explain

a busca, e a prova de qual índice ela usou

Fauxton

o painel das fichas e das versões

DESENVOLVIMENTO

Flask e Jinja

o cérebro e o molde

requests

o carteiro até o banco

python-dotenv

senhas fora do código

Git e GitHub

o histórico, com o projeto 1 guardado

QUALIDADE E OPERAÇÃO

pytest

109 testes

GitHub Actions

testa no CouchDB de verdade a cada envio

Node.js

roda a regra do banco fora dele, para testá-la

Vercel

publica o site a cada envio

Claude Code

CLAUDE OPUS 5 · ANTHROPIC

O **PARCEIRO DE BANCADA**. Assistente de programação com IA: escreveu código, testes, documentos e estes slides. As decisões foram do grupo — CouchDB, Vercel com Cloudant e o roteiro desta apresentação.

Sáímos do Railway por custo: Vercel e Cloudant, os dois no plano gratuito.

Modelagem documental

Cinco tipos de ficha: três principais e duas de apoio.

FICHA · PRODUTO

```
{
  "_id": "produto:chapada-geisha",
  "nome": "Chapada Geisha",
  "preco_centavos": 14800,
  "estoque": 1
}
```

FICHA · CLIENTE

```
{
  "_id": "cliente:3f2c...",
  "nome": "Ana",
  "email": "ana@exemplo.com",
  "senha_hash": "scrypt:..."
}
```

FICHA · PEDIDO

```
{
  "_id": "pedido:9b1d...",
  "cliente_id": "cliente:3f2c...",
  "status": "CRIADO",
  "itens": [{
    "nome": "Chapada Geisha",
    "preco_unitario_centavos": 14800,
    "moagem": "FINA",
    "quantidade": 1
  }]
}
```

A moagem continua no item: o cliente escolhe na compra.

EMBUTIR

os itens moram dentro do pedido

Grampeados na ficha: são lidos e alterados juntos.

REFERENCIAR

o pedido anota o cliente_id

Como anotar o número da ficha do cliente: ele tem vida própria.

FOTOGRAFAR

nome e preço copiados no item

Uma foto do dia da compra: o preço de amanhã não muda o pedido de hoje.

Fichas de apoio: categoria (a região) e email:<endereço>, que garante e-mail único. Dinheiro em centavos: o JSON não tem número decimal exato.

O que mudou em relação ao PostgreSQL

Os dois guardam regras. A diferença real é o “tudo ou nada”.

POSTGRESQL · PROJETO 1

Um cofre com regras e uma tranca para o pedido inteiro.

COUCHDB · PROJETO 2

Um arquivo de fichas com um conferente na porta: confere cada ficha sozinha, mas não enxerga o conjunto.

	POSTGRESQL	COUCHDB
Estrutura	5 tabelas ligadas	fichas JSON completas
Identidade	número automático: 1, 2, 3...	etiqueta escolhida: produto:chapada-geisha
Relacionamento	chave estrangeira e JOIN	embutir, referenciar ou fotografar
Consulta	SQL	Mango: um JSON enviado por HTTP
Índice	CREATE INDEX	um índice para cada pergunta
Regras	CHECK: o cofre recusa	validate_doc_update: o conferente recusa
Concorrência	FOR UPDATE: tranca a gaveta	_rev: o número da versão; quem chega atrasado recebe 409
Tudo ou nada	a transação inteira	uma ficha de cada vez

O checkout — o coração do trabalho

No CouchDB não existe “tudo ou nada” entre fichas diferentes.

O RISCO

O pedido grava, a baixa de estoque falha: sobra pedido sem estoque descontado.

O PROJETO-BASE DO MATERIAL

Manda tudo num lote só e, se der conflito, só avisa o cliente. O que já foi gravado fica gravado.

1

Conferir

cafés e estoque, sem gravar nada

2

Anotar

o pedido nasce “processando”

3

Reservar

baixa o estoque e prende um bilhete do pedido na ficha do café

4

Confirmar

o pedido vira “criado” e não volta atrás

5

Limpar

tira os bilhetes

FALHOU NO 3 OU NO 4?

Cancela o pedido e estorna o estoque — só onde houver bilhete.

CONFLITO 409 NO 3?

Relê a ficha do café e tenta de novo.

Clique duplo não compra duas vezes: o número do pedido nasce quando a tela abre, e o banco não aceita dois pedidos com o mesmo número.

O teste de conflito

Duas compras disputando o último Chapada Geisha — forçadas de propósito, no instante exato.

- 1 **A Ana confere o carrinho** — Geisha: estoque 1, versão 1. Piatã: estoque 10.
- 2 **Outra compra chega antes** — leva o último Geisha, e a ficha vira versão 2
- 3 **A Ana reserva com a versão velha** — o Piatã entra; o Geisha volta com 409: “essa ficha mudou desde que você leu”
- 4 **A loja relê o Geisha** — estoque 0
- 5 **Estorno** — o pedido da Ana é cancelado, e o Piatã volta a 10
- 6 **A Ana vê a mensagem** — “Estoque insuficiente de Chapada Geisha”

RESULTADO REAL

Passou no Apache CouchDB 3.5.2 de verdade, no GitHub Actions.

`test_cafe_que_acaba_no_meio_da_saga_e_compensado`

SE AINDA SOBRA ESTOQUE

A loja só relê e tenta de novo: $10 - 4$ da outra compra $- 3$ desta = 3. Nenhuma baixa se perde.

`test_conflito_409_e_resolvido_relendo_o_documento`

Problemas encontrados e como resolvemos

Quatro problemas de verdade, cada um com a solução testada.

1

O checkout do projeto-base deixava pedido pela metade

Um conflito no meio do lote avisava o cliente, mas não desfazia o que já tinha entrado.

Saga com estorno, e um teste para cada jeito de falhar.

2

A máquina do grupo não tem Docker

O material sobe o CouchDB com Docker. Sem ele, não havia banco para testar.

Uma maquete do banco para testar em qualquer lugar, e o CouchDB de verdade no GitHub Actions a cada envio.

3

O CouchDB não tem "e-mail único"

Consultar antes de gravar não basta: dois cadastros ao mesmo tempo passariam juntos.

Uma ficha cuja etiqueta é o próprio e-mail. Duas fichas com a mesma etiqueta não existem.

4

A resposta se perdeu: gravou ou não?

Se a conexão cai no meio da reserva, a loja não sabe o que o banco guardou.

O bilhete de reserva na ficha do café diz exatamente o que entrou.

E dois menores: a pasta nova ficava dentro da antiga e lia a senha errada · uma regra do banco que nunca era usada, achada rodando a função no Node.

0 projeto em números

109

testes

todos passando no CouchDB de verdade

35

regras

conferidas pelo próprio banco

8

idas ao banco

para confirmar uma compra, com um café ou com vários

4

índices

um para cada pergunta

5

tipos de ficha

3 principais e 2 de apoio

16

fichas

carregadas num lote só: 4 regiões e 12 cafés

O QUE FICOU DE FORA, DE PROPÓSITO

- Pagamento e frete
- Área administrativa
- Carrinho guardado no banco — ele fica na sessão
- Reconciliação automática — hoje é um comando
- Mais de 50 pedidos por página

[PREENCHER: tempo de resposta no ar, medido em /saude depois do deploy]

Demonstração

projeto-nivaldo-nosql.vercel.app

- 1 Catálogo e filtro por região
- 2 Escolher café e moagem — a moagem vai para o item do pedido
- 3 Criar conta e comprar: o pedido aparece com o preço do dia
- 4 Pôr o Chapada Geisha duas vezes no carrinho e finalizar: a loja diz qual café faltou**
- 5 No painel do Cloudant: a ficha do pedido e a versão do café mudando

O Chapada Geisha está com estoque 1, de propósito.



Estoque do Torra & Terra

Aponte a câmera do celular e fale com
a gente no WhatsApp.

+55 19 99734-7157